

The while loop — the program that insists

TC1037 · Algorithmic Thinking · Lecture 05

Dr. Leopoldo Rodríguez Salazar

Tuesday 6 October 2026

Tecnológico de Monterrey · Campus Querétaro · Group 510

Two of them send at the same instant.

Both messages are garbage.

What should a laptop do next?

Algorithms to Live By, ch. 10 — *Networking*

Exponential backoff (1973)

After each collision, wait a **random** time
in a range that **doubles**: 0–1, 0–3, 0–7, 0–15, ...

Give up after 16 tries.

- It **repeats while** a condition holds — *not sent yet*.
- It has a **step** that changes every turn — the range doubles.
- It has an **exit you can prove** — sent, or 16 attempts. Nothing can keep it running forever.

Send with backoff, in our notation.

INPUT: the message

OUTPUT: sent — or gave up

Numbered steps. You will need:

- a counter for the attempts
- a variable for the waiting range

Circle one step

The step that decides whether to go round again.

That circled step is the only new word Python learns today.

The line we could not write

On the exam: *if the consumption is negative, display a message and stop. Stop?*

```
1 cubic_meters = float(input("Consumption (m3): "))
2 while cubic_meters < 0:
3     print("Consumption cannot be negative. Try again.")
4     cubic_meters = float(input("Consumption (m3): "))
5 print("Thank you. Billing", cubic_meters, "m3.")
```

while, not **if**: after the block, **go back to the test**.

```
1 count = 1                # start
2 while count <= 5:         # test
3     print(count)
4     count = count + 1     # step
5 print("Done. count is now", count)
```

Predict the last line.

The loop ends when the test *fails*. The counter is always one step past the last thing printed.

Reading until a sentinel

```
1 total = 0
2 count = 0
3 speed = float(input("Speed (0 to finish): "))
4 while speed != 0:
5     total = total + speed
6     count = count + 1
7     speed = float(input("Speed (0 to finish): "))
8 if count > 0:
9     print("Count:", count, "Avg:", total / count)
10 else:
11     print("No measurements.")
```

The idiom: one `input()` before the loop, one as the *last line inside* it. **The accumulators:** start at 0, grow one step per turn. **Zero times:** the user may type 0 first — 0 / 0.

The loop that never ends

```
1 n = 10
2 while n > 0:
3     print("n is", n)
4     # n = n - 1
```

X No error. No wrong output. No end.

Find the test. Find the step. The step is missing. **Ctrl+C**.

✓ How do you know it stops?

Name the quantity that moves toward the exit **every** turn. If you cannot name it, you are hoping.

Counting with while — and the order of the step

```
total = 0
i = 1
while i <= n:
    total = total + i
    i = i + 1
```

$n = 10 \rightarrow 55$

```
total = 0
i = 1
while i <= n:
    i = i + 1
    total = total + i
```

$n = 10 \rightarrow 65$

Same five lines. The order of the step inside the block changes what the loop does.

Also: $n = 0 \rightarrow 0$ (block never runs). $n = -3 \rightarrow 0$. Is that right? *The spec did not say.*

Three accumulators at once

```
maximum = None                                # "no maximum yet"
...
    if speed < 0:
        print("Negative speed ignored.")
    else:
        total = total + speed
        count = count + 1
        if maximum is None or speed > maximum:
            maximum = speed
```

Last week's `if` lives *inside* this week's `while`, four spaces deeper. Blocks nest. Read the indentation.

A loop inside a function — Friday's lab shape

```
def positive_sum(how_many):
    total = 0
    asked = 0
    while asked < how_many:
        value = float(input("Number: "))
        if value > 0:
            total = total + value
            asked = asked + 1
    return total

def main():
    how_many = int(input("How many? "))
    print("Positive sum:", positive_sum(how_many))
```

Shell: `positive_sum(0) → 0` **without asking anything**. That test catches `<=`.

Three loops that run — and are wrong

X 1 · step outside the block

```
i = 1
while i <= 3:
    print("A", i)
i = i + 1
```

Prints A 1 forever.

X 2 · off by one

```
i = 0
while i < 5:
    i = i + 1
    print("B", i)
```

1..5. Move the print up: 0..4.

Read the spec.

X 3 · the sentinel is data

If 0 ends the loop, nobody can enter a real 0.

Choose a sentinel outside the data's range — or the design is wrong before any Python exists.

In-class activity 2 — speeds, until you stop · 20 min · graded

1. Read speeds (km/h) one at a time, **until the user enters 0**.
2. A **negative** speed: display Negative speed ignored. and keep going.
3. At 0: how many measurements, the **average**, the **maximum**.
4. No measurements: No measurements. — and nothing else.

you type	expected
100 90 140 0	3 · avg 110.0 · max 140.0
0	No measurements.
-5 60 0	ignored · 1 · 60.0 · 60.0
80 80 0	2 · 80.0 · 80.0

Algorithm on paper first — WHILE and IF lines written out; **circle the step that stops it**. Program in exam format. Tests under the algorithm.

Submit: photo + A_Matricula_ic2.py → Canvas → *In-class activity 2: while loops* — Sunday 11 Oct, 23:59.

Rubric: algorithm 40 · four tests 40 · zero case + negatives 20.

Before Friday

1. **Homework 4** — `collatz_steps`, `reverse_digits`, a `main`; every loop with its *why it stops* sentence. Due **Sunday 11 Oct, 23:59**.
2. **Levelling module closes Thursday 8 Oct, 23:59**. All ten lessons. Lessons 9–10 are today's lecture drawn as diamonds.
3. **Friday 16:30, Lab 3**: the **final diagnostic** (20 min, laptop, 1 %) then the academia's `while` lab.
4. Friday's session: four loops as functions, tested at the boundary. Reread *start · test · step*.

Quick exam 3: Tuesday 13 October, first 20 minutes.

The `while` loop — `read · predict · fix`, plus one loop to write.

while not done:

find the test. find the step. name what moves.